👨‍🏫 LA SOLUCIÓN ORIGINAL DE AULA

Solución 0: Enfoque de Clase de Ramón Abramo

✍️ Por Ramón Abramo • Profesor de Desarrollo Web

¡Muy buenas a todos! Soy Ramón Abramo. En esta guía detallada os presento la **Solución 0**, exactamente la que construimos y analizamos juntos en la pizarra del aula. Se caracteriza por su tremenda claridad pedagógica: agrupamos los botones en tres listas naturales (`numeros` , `operadores` y `acciones`) y creamos una función pura de cálculo `calcular(numero1, numero2, operacion)` .

🎯 1. ¿Por qué este enfoque triunfa en clase?

Cuando explicamos JavaScript en la pizarra, los alumnos necesitan ver una correspondencia directa entre lo que ven en el HTML y el bloque de código que lo maneja:

🔢 1. Teclas 'numeros'
Un bucle para los dígitos: si hay un `'0'` o venimos de pulsar operador, sustituye; si no, concatena.

➕ 2. Teclas 'operadores'
Un bucle para +, -, *, /: calcula el subtotal si ya había operación pendiente y memoriza la nueva.

🎯 3. Teclas 'acciones'
Un bucle para `igual` y `borrar` con un `switch` limpio y conciso.

📂 Archivos de esta Carpeta (`version-0-solucion-clase/`)

- `app.solucion.js` : La solución de clase idéntica a la que subí (antiguo `app1.js`).
- `app.js` : Código maestro paso a paso con comentarios y pruebas del alumno.
- `app.alumnos.js` : Plantilla con bloques `TODO` vacíos para entregar en clase.
- `presentacion.html` : Diapositivas interactivas para proyectar en el aula.
- `GUIA_PASO_A_PASO.md` : Guía rápida en Markdown.

1. La Función Pura: `calcular(numero1, numero2, operacion)`

En mi solución de clase, la función de cálculo es **pura**: no depende de variables globales ni modifica el DOM directamente. Recibe los dos números y la operación, y devuelve el resultado numérico:

```
function calcular(numero1, numero2, operacion) {
    let resultado = 0;

    switch (operacion) {
        case '+':
            resultado = numero1 + numero2;
            break;
        case '-':
            resultado = numero1 - numero2;
            break;
        case '*':
            resultado = numero1 * numero2;
            break;
        case '/':
            if (numero2 === 0) {
                resultado = 'Error';
            } else {
                resultado = numero1 / numero2;
            }
            break;
    }

    return resultado;
}
```

💡 **La Belleza de una Función Pura**

Al escribir `calcular(15, 3, '+')`, esta función devuelve `18` sin tocar la pantalla ni ninguna variable externa. Esto permite reutilizarla tanto al pulsar el botón `=` como al encadenar operadores (ej. `5 + 3 + ...`).

⚠️ **Control de División por Cero**

Fijaos en la línea `if (numero2 === 0) resultado = 'Error';`. Si intentamos dividir entre 0, devolvemos el texto `'Error'`. Más adelante veréis cómo este texto bloquea que se sigan haciendo operaciones erróneas.

2. Las 4 Variables de Estado de Clase

> En la clase definimos cuatro variables globales con nombres muy intuitivos que describen el flujo de pensamiento de un usuario:

Variable	Valor Inicial	Función en la Calculadora
`empezarNumero`	`false` (booleano)	Pasa a `true` tras pulsar un operador para que el próximo dígito limpie la pantalla.
`operadorPendiente`	`null` (string)	Almacena el operador (`'+'`, `'-'`, etc.) que está esperando a resolverse.
`numeroAnterior`	`null` (número)	El número que había en pantalla antes de presionar el operador matemático.
`numeroActual`	`null` (número)	El número que hay en pantalla en el momento de ejecutar el cálculo.

3. Selección en el DOM: Las 3 Listas Semánticas

Seleccionamos directamente los tres grupos de botones y la pantalla de salida:

```
const numeros = document.querySelectorAll('[data-numero]');
const operadores = document.querySelectorAll('[data-operador]');
const acciones = document.querySelectorAll('[data-accion]');
const pantalla = document.querySelector('output');
```

> 📌 **¿Por qué 'acciones' agrupa a Igual y Borrar?**
> En el HTML, tanto la tecla `=` como la tecla `C` tienen el atributo `data-accion` (con valores `"igual"` y `"borrar"`). Con un solo selector capturamos ambas teclas de control.

[HITO 1] El Bloque de Dígitos: `numeros.forEach`

Recorremos la lista `numeros` y asignamos el escuchador click. Observad la lógica de sustitución y concatenación directa sobre `pantalla.textContent`:

```
numeros.forEach(numero => {
    numero.addEventListener('click', () => {
        // 1. Leemos el dígito pulsado desde el dataset
        const valor = numero.dataset.numero;

        // 2. Si la pantalla tiene '0' o venimos de pulsar un operador:
        if (pantalla.textContent === '0' || empezarNumero) {
            pantalla.textContent = valor; // Escribe el nuevo número limpio
            empezarNumero = false;        // Desactiva la bandera
        } else {
            // 3. Si no, pega el dígito detrás de los existentes
            pantalla.textContent += valor;
        }
    });
});
```

Explicación de la Lógica Paso a Paso

Caso A: El 0 Inicial

Al abrir la calculadora, la pantalla tiene `'0'`. Si pulsamos `7`, `pantalla.textContent === '0'` es `true`, por lo que se sustituye por `"7"` (evitamos que aparezca `"07"`).

Caso B: El Flag 'empezarNumero'

Si el usuario teclea `15` y pulsa `+`, `empezarNumero` se activa en `true`. Al teclear el siguiente número `4`, la pantalla se limpia automáticamente mostrando solo `"4"`.

🍼 **Prueba de Fuego del Alumno**

Pulsad `7`, `8`, `9`. En pantalla se ve `789`. La concatenación y la eliminación del cero funcionan perfectamente.

[HITO 2, 4 y 5] Operadores (+, -, *, /)

> Este es uno de los bloques más inteligentes de la solución de clase: resuelve las operaciones encadenadas automáticamente sin añadir código extra:

```
operadores.forEach(operador => {
    operador.addEventListener('click', () => {
        const valor = operador.dataset.operador;

        // [HITO 5] Si ya había operador pendiente y ya se escribió un número:
        if (operadorPendiente !== null && empezarNumero !== true) {
            numeroActual = Number(pantalla.textContent);
            numeroAnterior = calcular(numeroAnterior, numeroActual, operadorPendiente);
            pantalla.textContent = numeroAnterior; // Pinta el subtotal en pantalla
        } else {
            // Si es el primer operador pulsado:
            numeroAnterior = Number(pantalla.textContent);
        }

        // Si hubo error (ej. división por 0), no guardamos el operador
        operadorPendiente = (numeroAnterior === 'Error') ? null : valor;

        // Marcamos que el próximo dígito debe limpiar la pantalla
        empezarNumero = true;
    });
});
```

Los Dos Escenarios de Pulsar un Operador

Escenario 1: Operación Simple

El usuario teclea `12` y pulsa `+`. Como `operadorPendiente` era `null`, entra en el `else`: guarda `numeroAnterior = 12`, `operadorPendiente = '+'` y `empezarNumero = true`.

Escenario 2: Operación Encadenada

El usuario teclea `5 + 3` y luego pulsa `*`. Como ya había un `+` pendiente y escribió el `3`, se calcula `5 + 3 = 8`, se pinta `8` en pantalla y se guarda `*` como nuevo operador.

[HITO 3 y 6] Las Acciones: Igual (=) y Borrar (C)

> Recorremos la lista `acciones` y despachamos con un `switch (valor)`:

```
acciones.forEach(accion => {
    accion.addEventListener('click', () => {
        const valor = accion.dataset.accion;

        switch (valor) {
            case 'igual':
                // Si hay una operación pendiente, resolvemos el cálculo
                if (operadorPendiente !== null) {
                    numeroActual = Number(pantalla.textContent);
                    numeroAnterior = calcular(numeroAnterior, numeroActual, operadorPendiente);
                    pantalla.textContent = numeroAnterior;
                }
                operadorPendiente = null;
                empezarNumero = true;
                break;

            case 'borrar':
                pantalla.textContent = '0';
                operadorPendiente = null;
                empezarNumero = true;
                break;
        }
    });
});
```

Resumen Didáctico de la Solución 0

¿Qué hace única a la Solución de Clase?

1. **Función pura:** `calcular(n1, n2, op)` es un modelo ejemplar de modularidad.
2. **Manipulación directa:** Lee y escribe en `pantalla.textContent` sin capas intermedias.
3. **Semántica de grupos:** Dividir en `numeros`, `operadores` y `acciones` ayuda al alumno a estructurar mentalmente los tipos de botones.

¡Enhorabuena! Has completado el análisis de la Solución de Clase de Ramón Abramo. 🎓